5

Abstrações

Universidade Estadual da Paraíba (UEPB)
Paradigmas de Programação
Kláudio Henrique Mascarenhas Medeiros

Abstrações

- A abstração é uma **forma de gerenciar a complexidade em um sistema de software**, facilitando a sua compreensão e a comunicação com o cliente.

Abstrações

- A abstração é uma **forma de gerenciar a complexidade em um sistema de software**, facilitando a sua compreensão e a comunicação com o cliente.

- Podem ser aplicadas em análise de sistemas, bem como no projeto, implementação e testes destes sistemas.

Abstrações

- A abstração é uma **forma de gerenciar a complexidade em um sistema de software**, facilitando a sua compreensão e a comunicação com o cliente.

- Podem ser aplicadas em análise de sistemas, bem como no projeto, implementação e testes destes sistemas.

- As abstrações focam em ideias gerais, deixando de lado manifestações destas ideias.

Abstrações

Uma classe Pessoa em Java não terá todos os atributos e comportamentos de uma pessoa

Abstrações

Um método dessa classe também pode ser entendido como uma abstração

Abstrações em programação

- A abstração nos permite fazer distinção do que um trecho de programa faz e como foi implementado.

Abstrações em programação

- A abstração nos permite fazer distinção **do que um trecho de programa faz** e **como foi implementado**.

- Em muitas ocasiões, não precisamos saber detalhes de como a abstração foi implementada.

Abstrações

Linguagens declarativas são focadas no “o que”, imperativas mais focadas no “como”

Abstrações em programação

- A abstração nos permite fazer distinção **do que um trecho de programa faz** e **como foi implementado**.

- Em muitas ocasiões, não precisamos saber detalhes de como a abstração foi implementada.

- A abstração contém uma computação, podendo ser:
  - de **função**: c/ uma expressão a ser avaliada
  - de **procedimento**: c/ um comando a ser executado

Abstrações

```
procedure imprime_msg();
begin
    writeln('Essa abstração não retorna nada');
end;

function dobra(x: integer) : integer;
begin
    writeln('Essa abstração retorna um valor');
    dobra := 2 * x;
end;

begin
    imprime_msg();
    writeln(dobra(20));
end.
```

```
24 lines compiled, 0.1 sec
Essa abstração não retorna nada
Essa abstração retorna um valor
40
```

Abstração de função

- A abstração de função contém uma expressão a ser avaliada. Esta resultará em um valor.

Abstração de função

- A abstração de função contém uma expressão a ser avaliada. Esta resultará em um valor.

- Exemplo: T function F(DPF1, ..., DPFn) B
  - T entendido como tipo do valor retornado
  - F como sendo o nome da função
  - DPF como declarações de parâmetros formais
  - B como sendo o bloco contendo implementação

Abstração de função

- Exemplo: T function F(DPF1, ..., DPFn) B
  - T entendido como tipo do valor retornado
  - F como sendo o nome da função
  - DPF como declarações de parâmetros formais
  - B como sendo o bloco contendo implementação
- Exemplo de chamada de abstração: F(PR1, ..., PRn)
  - PR como sendo o parâmetro real (ou argumento).

Abstração de procedimento

- A abstração de procedimento é uma abstração sobre comando, focado em atualização de variáveis pela execução do corpo da abstração.

Abstração de procedimento

- A abstração de procedimento é uma abstração sobre comando, focado em atualização de variáveis pela execução do corpo da abstração.

- Exemplo: em Python, funções sem retorno são entendidas como procedimentos (retornam None).

```
def funcao(self): // não retorna nada, não explicita tipo de retorno
    self.ativado = True
    self.metodo()
```

Parametrização de abstrações

- A parametrização de abstrações é bastante útil, fazendo, por exemplo, com que uma abstração possa gerar resultados de computação variados.

Parametrização de abstrações

- A parametrização de abstrações é bastante útil, fazendo, por exemplo, com que uma abstração possa gerar resultados de computação variados.

- Exemplo: uma função power, que calcula o quadrado de um número, deve retornar 4 se eu quiser calcular o quadrado de 2, e 81 se quiser o quadrado de 9.

Parametrização de abstrações

- A parametrização de abstrações é bastante útil, fazendo, por exemplo, com que uma abstração possa gerar resultados de computação variados.

- Exemplo: uma função power, que calcula o quadrado de um número, deve retornar 4 se eu quiser calcular o quadrado de 2, e 81 se quiser o quadrado de 9.

- Podemos conseguir isso parametrizando a abstração.

Parametrização de abstrações

```
float power(float n)
{
        return n * n;
}

float num = 9;
float result = power(num);
```

- n é dito ser um parâmetro formal. O parâmetro formal é um identificador usado dentro da abstração.

Parametrização de abstrações

```
float power(float n)
{
        return n * n;
}

float num = 9;
float result = power(num);
```

- n é dito ser um **parâmetro formal**. O parâmetro formal é um identificador usado dentro da abstração.
- num é dito ser um **parâmetro real**. O parâmetro real é uma expressão que resulta no argumento.

Parametrização de abstrações

```
float power(float n)
{
        return n * n;
}

float num = 9;
float result = power(num);
```

- n é dito ser um **parâmetro formal**. O parâmetro formal é um identificador usado dentro da abstração.
- num é dito ser um **parâmetro real**. O parâmetro real é uma expressão que resulta no argumento.
- O **argumento** é um valor passado para a abstração.

Argumentos

- Os valores que podem ser argumentos de uma abstração podem variar entre linguagens.

Argumentos

- Os valores que podem ser argumentos de uma abstração podem variar entre linguagens.

- Exemplo: em Pascal, um array não pode ser argumento, mas um apontador ou valor primitivo pode.

Argumentos

- Os valores que podem ser argumentos de uma abstração podem variar entre linguagens.

- Exemplo: em Pascal, um array não pode ser argumento, mas um apontador ou valor primitivo pode.

- Em algumas linguagens, podemos passar funções como argumentos.

Parametrização de abstrações

```
function call(f: Function) : void{
    f();
}

function printPLP(): void{
    trace("PLP");
}

call(printPLP());
```

- Exemplo: abstrações de função em ActionScript. call() tem como parâmetro formal uma abstração

Parametrização de abstrações

- Quando a abstração é chamada, cada parâmetro formal é vinculado ao argumento correspondente.

Parametrização de abstrações

- Quando a abstração é chamada, cada parâmetro formal é vinculado ao argumento correspondente.

- É importante observar qual é o mecanismo de passagem de parâmetros que irá acontecer. Estes podem ser **por cópia** ou **por referência**.

Passagem de parâmetro p/ cópia

- Nesse tipo de passagem, os valores são copiados de ou para uma abstração, quando esta é chamada.

Passagem de parâmetro p/ cópia

- Nesse tipo de passagem, os valores são copiados de ou para uma abstração, quando esta é chamada.

- O parâmetro formal representa uma variável local da abstração, criada quando ela começa e é destruída quando ela acaba.

Passagem de parâmetro p/ cópia

- Nesse tipo de passagem, os valores são copiados de ou para uma abstração, quando esta é chamada.

- O parâmetro formal representa uma variável local da abstração, criada quando ela começa e é destruída quando ela acaba.

- Pode ser de **valor**, de **resultado** ou de **valor/resultado**.

Parametrização p/ cópia (valor)

- No início da abstração, é criada uma variável local L e o valor dela é o valor definido no argumento.

Parametrização p/ cópia (valor)

- No início da abstração, é criada uma variável local L e o valor dela é o valor definido no argumento.

- L (o parâmetro formal) é tratado como uma variável local.

Parametrização p/ cópia (valor)

- No início da abstração, é criada uma variável local L e o valor dela é o valor definido no argumento.
- L (o parâmetro formal) é tratado como uma variável local.
- O valor de L pode ser inspecionado e atualizado.

Parametrização p/ cópia (valor)

- No início da abstração, é criada uma variável local L e o valor dela é o valor definido no argumento.
- L (o parâmetro formal) é tratado como uma variável local.
- O valor de L pode ser inspecionado e atualizado.
- As atualizações de valor de L não apresentam resultado algum em variáveis não-locais.

Parametrização p/ cópia (resultado)

- Ela possui um argumento de entrada (uma variável indefinida).

Parametrização p/ cópia (resultado)

- Ela possui um argumento de entrada (uma variável indefinida).

- No início da abstração, é criada uma variável local L indefinida. O valor dessa variável também é indefinida.

Parametrização p/ cópia (resultado)

- Ela possui um argumento de entrada (uma variável indefinida).
- No início da abstração, é criada uma variável local L indefinida. O valor dessa variável também é indefinida.
- O valor de L pode ser inspecionado e atualizado.

Parametrização p/ cópia (resultado)

- Ela possui um argumento de entrada (uma variável indefinida).
- No início da abstração, é criada uma variável local L indefinida. O valor dessa variável também é indefinida.
- O valor de L pode ser inspecionado e atualizado.
- No final da abstração, o valor de L é atribuída para a variável argumento que foi recebida.

Parametrização p/ cópia (ambos)

- Esse tipo de parametrização combina os dois mencionados anteriormente:

Parametrização p/ cópia (ambos)

- Esse tipo de parametrização combina os dois mencionados anteriormente:
    - há a criação de uma variável local L definida.

Parametrização p/ cópia (ambos)

- Esse tipo de parametrização combina os dois mencionados anteriormente:
  - há a criação de uma variável local L definida.
  - L pode ser inspecionada e atualizada.

Parametrização p/ cópia (ambos)

- Esse tipo de parametrização combina os dois mencionados anteriormente:
  - há a criação de uma variável local L definida.
  - L pode ser inspecionada e atualizada.
  - ao final, o valor de L é atribuído à variável argumento que foi recebida pela abstração.

Passagem de parâmetro p/ cópia

- Linguagens como C permitem apenas passagem de parâmetros por cópia.

Passagem de parâmetro p/ cópia

- Linguagens como C permitem apenas passagem de parâmetros por cópia.

- Linguagens como Ada e Pascal permitem os três tipos de passagem de parâmetros por cópia (no caso de Ada, via in, out e inout).

Passagem de parâmetro p/ referência

- Nesse tipo de passagem, o parâmetro formal é associado diretamente com o argumento, não havendo a criação de variáveis locais.

Passagem de parâmetro p/ referência

- Nesse tipo de passagem, o parâmetro formal é associado diretamente com o argumento, não havendo a criação de variáveis locais.

- O parâmetro real pode ser uma:

  - constante: par. formal associado ao valor do arg.

Passagem de parâmetro p/ referência

- Nesse tipo de passagem, o parâmetro formal é associado diretamente com o argumento, não havendo a criação de variáveis locais.

- O parâmetro real pode ser uma:

  - **constante**: par. formal associado ao valor do arg.
  - **variável**: par. formal associado a uma variável

Passagem de parâmetro p/ referência

- Nesse tipo de passagem, o parâmetro formal é associado diretamente com o argumento, não havendo a criação de variáveis locais.

- O parâmetro real pode ser uma:

  - **constante**: par. formal associado ao valor do arg.
  - **variável**: par. formal associado a uma variável
  - **procedimento**: par. formal é um procedimento

Passagem de parâmetro p/ referência

- Nesse tipo de passagem, o parâmetro formal é associado diretamente com o argumento, não havendo a criação de variáveis locais.

- O parâmetro real pode ser uma:

  - **constante**: par. formal associado ao valor do arg.
  - **variável**: par. formal associado a uma variável
  - **procedimento**: par. formal é um procedimento
  - **função**: par. formal é uma função

Passagem de parâmetro p/ referência

- Em Pascal, é possível se realizar passagem por referência através de constantes e variáveis.

Passagem de parâmetro p/ referência

- Em Pascal, é possível se realizar passagem por referência através de constantes e variáveis.

- Em C, não se pode fazer passagem por referência de forma direta, mas pode-se simular via ponteiros.

Passagem por referência em C++

```cpp
#include <iostream>
using namespace std;

float square_by_val(float);
float square_by_ref(float&);
float square_by_val(float x) {
    x = x * x;
    return x;
}
float square_by_ref(float &x){
    x = x * x;
    return x;
}
```

```cpp
int main(){
    float n;
    n = 9;
    cout << "n antes de square_by_val: " << n << endl;
    square_by_val(n);
    cout << "n depois de square_by_val: " << n << endl;
    n = 9;
    cout << "n antes de square_by_ref: " << n << endl;
    square_by_ref(n);
    cout << "n depois de square_by_ref: " << n << endl;
}
```

```
n antes de square_by_val: 9
n depois de square_by_val: 9
n antes de square_by_ref: 9
n depois de square_by_ref: 81

--------------------------------
Process exited after 0.03966 seconds with return value 0
Pressione qualquer tecla para continuar. . .
```

Unidades de programa

- As **unidades de programa** podem representar subpartes de um programa, sendo fruto da decomposição de programas grandes.

Unidades de programa

- As **unidades de programa** podem representar subpartes de um programa, sendo fruto da decomposição de programas grandes.

- As unidades de programa estão relacionadas com abstrações.

Unidades de programa

- As **unidades de programa** podem representar subpartes de um programa, sendo fruto da decomposição de programas grandes.

- As unidades de programa estão relacionadas com abstrações.

- As unidades de programa têm vantagens como serem reutilizáveis e modificáveis, bem como propósito único e interface reduzida com outras unidades.

Unidades de programa

- As abstrações de funções e procedimentos são exemplos simples de unidades de programa.

- Podemos também ter:

  - pacotes: possuem declarações de vários componentes (e.g. tipos, constantes, variáveis, funções, sub-pacotes.

Unidades de programa

- As abstrações de funções e procedimentos são exemplos simples de unidades de programa.

- Podemos também ter:

  - **pacotes**: possuem declarações de vários componentes (e.g. tipos, constantes, variáveis, funções, sub-pacotes.

  - **tipos abstratos**: tipos com representações privadas, mas com operações públicas

Unidades de programa

- As abstrações de funções e procedimentos são exemplos simples de unidades de programa.

- Podemos também ter:

  - **pacotes**: possuem declarações de vários componentes (e.g. tipos, constantes, variáveis, funções, sub-pacotes.

  - **tipos abstratos**: tipos com representações privadas, mas com operações públicas

  - **classes**: definem estrutura de famílias de objetos, sendo os objetos conjuntos de variáveis privadas com operações públicas

Unidades de programa (pacote)

```
package physics is
    gravity: constant Float := 9.81
    G: constant Float := 6.7e-11;
end physics;
...
physics.G; (acesso ao valor do componente G de physics)
```

- Exemplo: definição de pacote physics em Ada

Unidades de programa (TAD)

- Um **tipo abstrato de dados (TAD)** é um tipo definido por um grupo de operações (constantes, funções e procedimentos).

Unidades de programa (TAD)

- Um **tipo abstrato de dados (TAD)** é um tipo definido por um grupo de operações (constantes, funções e procedimentos).

- Os TADs tem representação escondida do usuário, podendo ter representação trocada sem que usuário precise alterar seus programas.

Unidades de programa (TAD)

Em linguagens OO, por exemplo, se não mudarmos a “interface” dela, podemos mexer em detalhes internos sem isso afetar muito o código externo

Unidades de programa (TAD)

- Um **tipo abstrato de dados (TAD)** é um tipo definido por um grupo de operações (constantes, funções e procedimentos).

- Os TADs tem representação escondida do usuário, podendo ter representação trocada sem que usuário precise alterar seus programas.

- Podem ser providas operações de construção e destruição.

Unidades de programa (TAD)

```
class Ponto2D
{
    public:
        Ponto2D(float newX, float newY)
        {
            x = newX;
            y = newY;
        }

        ~Ponto2D()
        {
            cout << "Ponto2D(" << getX() << ", " << getY() << ") destruido." << endl;
        }
```

Unidades de programa (objeto)

- Um objeto é um tipo de unidade de programa formado por variáveis escondidas e operações sobre essas variáveis.

Unidades de programa (objeto)

- Um objeto é um tipo de unidade de programa formado por variáveis escondidas e operações sobre essas variáveis.

- As variáveis, idealmente, são acessadas apenas através de operações públicas.

Unidades de programa (objeto)

- Um objeto é um tipo de unidade de programa formado por variáveis escondidas e operações sobre essas variáveis.

- As variáveis, idealmente, são acessadas apenas através de operações públicas.

- A representação das variáveis pode ser modificada, sem que essa mudança exija alterações externas.

Unidades de programa (classe)

- Uma classe especifica o estado e comportamento de um conjunto de objetos similares entre si.

Unidades de programa (classe)

- Uma classe especifica o estado e comportamento de um conjunto de objetos similares entre si.

- Os objetos em questão possuem as mesmas variáveis e as mesmas operações.

Unidades de programa (classe)

- Uma classe especifica o estado e comportamento de um conjunto de objetos similares entre si.

- Os objetos em questão possuem as mesmas variáveis e as mesmas operações.

- Os componentes que são variáveis são chamados de variáveis de instância e suas operações são chamadas de construtores ou métodos.

Referências

SEBESTA, Robert M. **Conceitos de Linguagens de Programação**. Bookman, 2018.